

第二章 线性表

学习目的要求：

1. 线性表的定义和线性表的特征。
2. 线性表的顺序存储结构及其算法的实现。
3. 线性链表的描述及其算法的实现。
4. 循环链表和双向循环链表的描述。
5. 数组的顺序存储和矩阵的压缩存储的描述。



第二章 线性表

2.1 线性表的基本概念

2.2 线性表的顺序存储结构及其算

法

2.3 线性表的链式存储结构及其运

算

2.4 算法应用举例

2.5 数

组



2.2.1 线性表的基本概念

线性表 (linear list) 是由 n 个数据元素组成的有限序列。记作 $(a_1, a_2, \dots, a_i, \dots,$

线性表可以用一个标识符来命名, 如果用 A 来表示线性表, 则: $A = (a_1, a_2, \dots, a_i, \dots, a_n)$

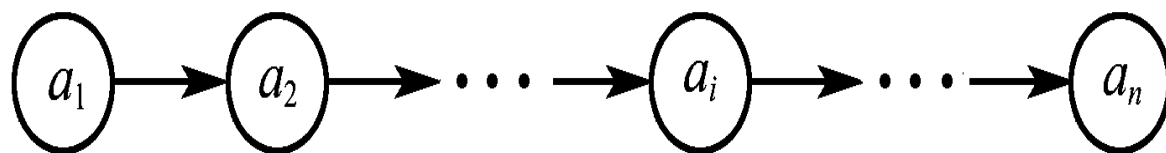


各数据元素的类型可以是程序设计语言中提供任何一种。



在线性表中, 各元素之间存在着线性的逻辑关系。

2.2.1 线性表的基本概念



线性表是一种非常典型的线性结构，用二元组可以表示成：

$$S = (D, R)$$

$$D = \{ a_1, a_2, \dots, a_i, \dots, a_n \}$$

$$R = \{ \langle a_1, a_2 \rangle, \langle a_2, a_3 \rangle, \dots, \langle a_i, a_{i+1} \rangle,$$

$\dots,$

$$\langle a_{n-1}, a_n \rangle \}$$



2.1.2 线性表的基本操作

2.1 线性表的基本概念

(1) InitList (List) : 初始化操作, 建立一个空的线性表 List;

(2) ListLength (List) : 求线性表的长度;

(3) GetElement (List , i) : 取线性表中的第 i 个元素 ($1 \leq i \leq n$, n 为线性表长度);

(4) PriorElement (List , x) : 若 x 不是第一个数据元素, 则取 x 的直接前驱;

(5) NextElement (List , x) : 若 x 不是最后一个元素, 则取 x 的直接后继;



2.1.2 线性表的基本操作

2.1 线性表的基本概念

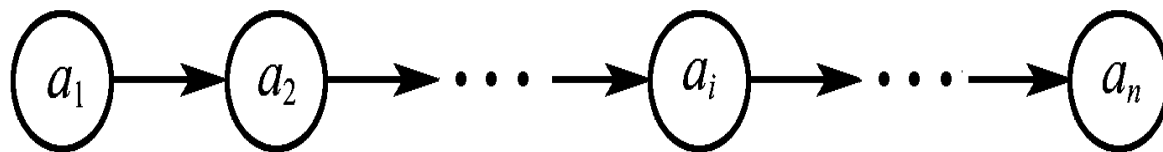
(6) LocateElement (List , x) : 若 x 存在于表中, 则取得 x 的位置 (位序) ;

(7) ListInsert (List , i , x) : 在线性表第 i 个元素之前插入一个数据元素 x ;

(8) ListDelete (List , i) : 删除线性表中第 i 个元素 ($1 \leq i \leq n$, n 为线性表长度) 。



及其算法



线性表的顺序存储结构简称为顺序表 (Sequential List)。

顺序存储结构的特点是：在线性表中逻辑关系相邻的数据元素，在计算机的内存中物理位置也是相邻的。

线性表中第 i 个元素 a_i 的存储位置


$$LOC(a_i) = LOC(a_1) + (i-1)L$$

$LOC(a_1)$	a_1	1
$LOC(a_1) + L$	a_2	2
	$\vdots$	$\vdots$
$LOC(a_1) + (i-1)L$	a_i	i
	$\vdots$	$\vdots$
$LOC(a_1) + (n-1)L$	a_n	n

2.2.1 线性表的顺序存储结构及其算法

线性表的顺序存储结构用 C 语言描述如下：

```
#define MAXLEN 100 /* 定义的大小与线性表的数据  
                    元素个数相关 */  
  
typedef int elementtype;  
          /* elementtype 可根据需要定义成其他类型 */  
  
typedef struct /* 定义线性表 */  
{ elementtype s[MAXLEN]; /* 定义线性表中的元素 */  
  int len; /* 定义线性表的表长 */  
}SqList;
```



2.2 线性表的顺序表示

2.2.2 顺序表的插入运算

1. 插入运算

插入运算是指在具有 n 个元素的线性表的第 i ($1 \leq i \leq n$) 个元素之前插入一个新元素 x , 使表长加 1。



插入原理

序号	数据
1	a_1
2	a_2
$\vdots$	$\vdots$
i	a_i
$i+1$	a_{i+1}
$\vdots$	$\vdots$
n	a_n
$\vdots$	$\vdots$
m	

插入前

序号	数据
1	a_1
2	a_2
$\vdots$	$\vdots$
i	x
$i+1$	a_i
$\vdots$	$\vdots$
$n+1$	a_n
$\vdots$	$\vdots$
m	

插入后

2.2 线性表的顺序存储结构及其算法

假如有一线性表：(1,2,3,4,5,6,7,8) 表长 len=8
在第 3 个元素之前插入一个新元素 9, 即
(1,2, 9 ,3,4,5,6,7,8) 表长 len=9

程序 2-1:

```
#include <stdio.h>
#define MAXLEN 100
typedef int elementtype;
typedef struct          /* 定义线性表 */
{ elementtype s[MAXLEN]; /* 定义线性表中元素 */
  int len;               /* 定义线性表的表长 */
}SqList;
```



2.2 线性表的顺序存储结构及其算法

```
int insertsqlist(int i,elementtype x,SqList *sql)
{   int j;
    if((i<1)||(i>sql->len)) /*i 值非法, 返回值为 0*/
        return(0);
    else
    {for(j=sql->len;j>=i;j--)
        sql->s[j+1]=sql->s[j];
        /* 向后移动数据, 腾出要插入的空位 */
        sql->s[j+1]=x; /* 修正插入位置为 j+1 */
        (sql->len)++; /* 表长加 1 */
        return(1); /* 插入成功, 返回值为 1 */
    }
}
```



2.2 线性表的顺序存储结构及其算法

```
main()                                /* 主程序 */
{ int b=3,c,k;      elementtype d=9;
  SqList a={0,1,2,3,4,5,6,7,8};
    /* 赋线性表各元素初值, a.s[0] 闲置不用 */
  a.len=8;                            /* 赋线性表长度值 */
  for(k=1;k<=a.len;k++)
  {printf("%3d",a.s[k]); /* 输出插入前结果 */
  }
  printf("\n");
  c=insertsqlist(b,d,&a); /* 调用插入函数 */
```



2.2 线性表的顺序存储结构及其算法

```
if(c==0)
    printf("error");
else
    {for(k=1;k<=a.len;k++)
        printf("%3d",a.s[k]);    /* 输出插入后结果
    */
    }
    printf("\n");
}
```



平均移动元素次数: $n/2$



2.2 线性表的顺序存储结构及其算法

2. 删除运算

删除运算是指从具有 n 个元素的线性表中，删除其中的第 i ($1 \leq i \leq n$) 个元素，使表的长度减 1。

删除原理



序号	数据
1	a_1
2	a_2
$\vdots$	$\vdots$
i	a_i
$i+1$	a_{i+1}
$\vdots$	$\vdots$
n	a_n
$\vdots$	$\vdots$
m	

删除前

序号	数据
1	a_1
2	a_2
$\vdots$	$\vdots$
i	a_{i+1}
$i+1$	a_{i+2}
$\vdots$	$\vdots$
$n-1$	a_n
$\vdots$	$\vdots$
m	

删除后



2.2 线性表的顺序存储结构及其算法

假如有一线性表：(1,2,3,4,5,6,7,8) 表长 len=8
删除第 4 个元素，即 (1,2, 3, 5,6,7,8) 表长 len=7

程序 2-1:

```
#include <stdio.h>
#define MAXLEN 100
typedef int elementtype;
typedef struct          /* 定义线性表 */
{ elementtype s[MAXLEN]; /* 定义线性表中元素 */
  int len;               /* 定义线性表的表长 */
}SqList;
```



2.2 线性表的顺序存储结构及其算法

```
int delsqllist(int i, SqList *sql)
{ int j;
  if((i<1)||i>sql->len)) /*i 值非法, 返回值为 0*/
    return(0);
  else
  { for(j=i+1;j<=sql->len;j++)
      sql->s[j-1]=sql->s[j];
      /* 向前移动数据, 覆盖前一数据 */
    (sql->len)--; /* 表长度减 1 */
    return(1); /* 删除成功, 返回值为 1 */
  }
}
```



2.2 线性表的顺序存储结构及其算法

```
main()                                /* 主程序 */
{ int b=4,c,k;      elementtype d=9;
  SqList a={0,1,2,3,4,5,6,7,8};
    /* 赋线性表各元素初值, a.s[0] 闲置不用 */
  a.len=8;                          /* 赋线性表长度值 */
  for(k=1;k<=a.len;k++)
  {printf("%3d",a.s[k]); /* 输出插入前结果 */
    }
  printf("\n");
  c=delsqlist(b,&a); /* 调用插入函数 */
  if(c==0)
    printf("error");
```



2.2 线性表的顺序存储结构及其算法

```
else
{for(k=1;k<=a.len;k++)
    printf("%3d",a.s[k]);    /* 输出插入后结果 */
}
printf("\n");
}
```

平均移动元素次数： $(n-1)/2$

不足之处：对于顺序表而言，其插入和删除时都需要移动大量元素，所以耗时较多，时间复杂度为 $O(n)$ 。另外因其使用数组来存放各个数据元素，有可能造成一部分存储空间的浪费或数据溢出。这是由于顺序表存储结构决定的，即逻辑上相邻的数据元素在存放时物理位置也相邻。



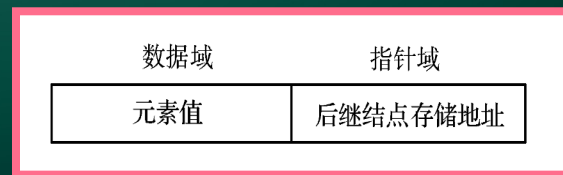
2.3 线性表的链式存储结构及其运算

2.3.1 线性表的链式存储结构

采用链式存储结构存储时它不要求逻辑上相邻的元素在物理位置上也相邻。

为了表示出每个元素与其直接后继元素之间的关系，除了存储元素本身的信息外，还需存储一个指示其直接后继元素的存储位置信息。存放数据元素的空间（节点）至少应包括两个域：数据域（存放数据元素值）、指针域（存放后继节点的存储地址）

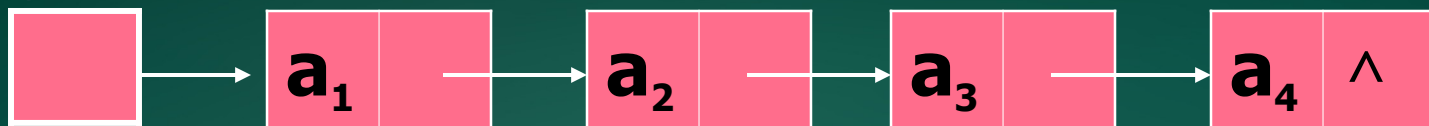
```
typedef struct node
{ int data;
  struct node *next;
}NODE;
```



2.3 线性表的链式存储结构及其运算

假定有线性表 (a_1, a_2, a_3, a_4) 。

Head



这些节点存储时可以存放至不连续的内存空间（链接指针）。
线性链表是通过结点指针域中的指针表示各结点之间的线性关系的。
另外，还需要设计一个表头指针，存放链表中第一个节点的存储地址。

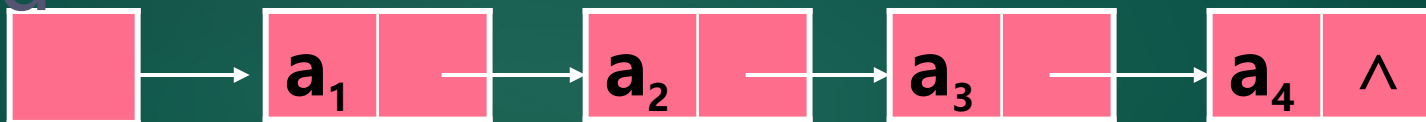
Head



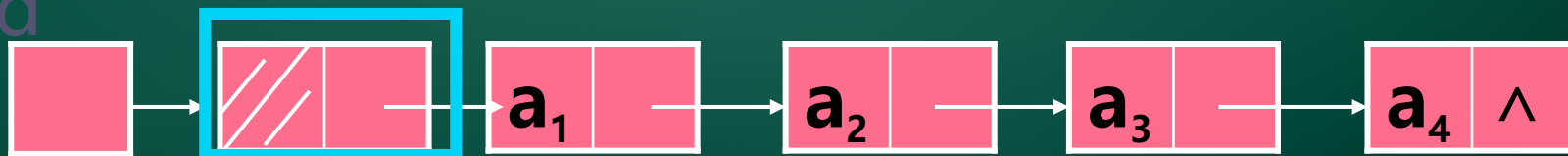
2.3 线性表的链式存储结构及其运算

为了便于实现各种运算，通常在链表的第一个节点之前设置一个附加节点，称之为表头节点，而其他节点称之为表中节点。表头节点结构与表中节点相同，只是数据域不存放数据元素。

Head



Head



若线性链表的每个结点只含有一个指针域，则称这样的线性链表为单链表。



2.3 线性表的链式存储结构及其运算

2.3.2 单链表的运算

1. 建立带表头结点的单链表

建立链表时，首先要建立表头结点，此时为空链表。然后采用循环方式将新的结点逐一插入到链表中，其过程如下：

- (1) 申请存储单元，用 C 语言的动态分配库函数 malloc 得到。
 - (2) 读入新结点的数据，新结点的指针域为空。
 - (3) 把新结点链接到链表上去（有前插和后插两种方式）。
- 重复以上步骤，直到将所有结点都链接到链表上为止。



2.3 线性表的链式存储结构及其运算

```
#include <stdio.h>
typedef struct node /* 定义单链表的节点 */
{ int data;
  struct node *next;
}NODE;
NODE *create() /* 采用后插入方式建立单链表 */
{ NODE *head,*q,*p; /* 定义指针变量 */
  char ch; int a;
  head=(NODE*)malloc(sizeof(NODE));
  /* 申请新的存储空间, 建立表头结点 */
  q=head;
  ch='*';
  printf("\nInput the list :");
}
```



(Continue ...)

2.3 线性表的链式存储结构及其运算

```
while(ch!='?')
    /*"ch" 为是否建立新结点的标志, 若 "ch" 为 "?" 则输入结束 */
    { scanf("%d",&a);          /* 输入新元素 */
      p=(NODE*)malloc(sizeof(NODE));
      p->data=a;
      q->next=p;
      q=p;
      ch=getchar();/* 读入输入与否的标志 */
    }
q->next=NULL;
return(head);    /* 返回表头指针 head*/
}
```



2.3 线性表的链式存储结构及其运算

```
main()
{  NODE *a;
   a=create();
   printf("output the list:");
   a=a->next;
   while(a!=NULL)
   {
       printf("%d ",a->data);    /* 输出链表各元素
*/
       a=a->next;
   }
```



2.3 线性表的链式存储结构及其运算

2. 单链表中结点的查找

从头指针开始查找。单链表有两种查找方法，即按给定值查找和按序号查找。

(1) 按值查找

按值查找，就是在单链表中查找是否存在数据域值为给定的值（如整数 x ）的结点。

(2) 按序号查找

在单链表中查找第 i 个位置上的结点，若找到，则返回它的地址，否则返回 NULL。



2.3 线性表的链式存储结构及其运算

(1) 按值查找

按值查找，就是在单链表中查找是否存在数据域值为给定的值（如整数 x ）的结点。

```
NODE *locate (NODE *head,int x)
/* 在已知链表中查找给定的值 x */
{
    NODE *p;
    p=head->next;
    while((p!=NULL)&&(p->data!=x))
        /* 未到表尾且未找到给定数据 */
        p=p->next;    /* 指向下一个元素 */
    return(p);
}
```



2.3 线性表的链式存储结构及其运算

(2) 按序号查找

在单链表中查找第 i 个位置上的结点，若找到，则返回它的地址，否则返回 NULL。

```
NODE *find(NODE *head,int i)
{ int j=1;
  NODE *p;
  p=head->next;
  while((p!=NULL)&&(j<i))
      /* 未到表尾且未找到给定数据 */
      { p=p->next;          /* 指向下一个元素 */
        j++;
      }
  return(p);
}
```

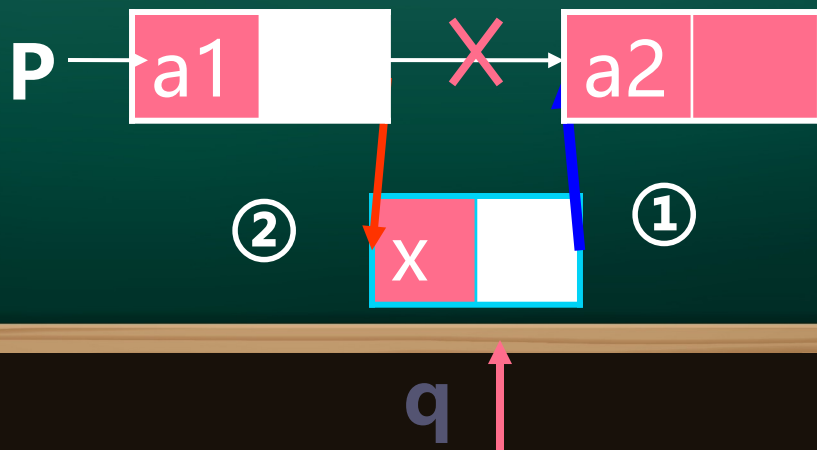


2.3 线性表的链式存储结构及其运算

3. 单链表上的插入运算

在顺序表中，插入运算时，将会有大量元素向后移动。而在单链表中，插入一个结点不需要移动元素，只需要修改指针即可。

若将 x 插入到 a_1 和 a_2 之间，其过程如下：（1）建立一个新结点 q ，将 x 值赋给 $q \rightarrow data$ 。（2）修改有关结点的指针域。



2.3 线性表的链式存储结构及其运算

```
void insert(NODE *p,int x)
    /* 在链表的 p 结点后插入给定元素 x */
{   NODE *q;
    q=(NODE*)malloc(sizeof(NODE));
                                   /* 申请新的存储空间 */
    q->data=x;
    q->next=p->next;              /* 实现图的① */
    p->next=q;
    /* 实现图的②，将新结点 q 链接到 p 结点之后 */
}
```



2.3 线性表的链式存储结构及其运算

4. 单链表上的删除运算

删除单向链表中的结点 x ，并由系统收回其占用的存储空间，其过程如下：

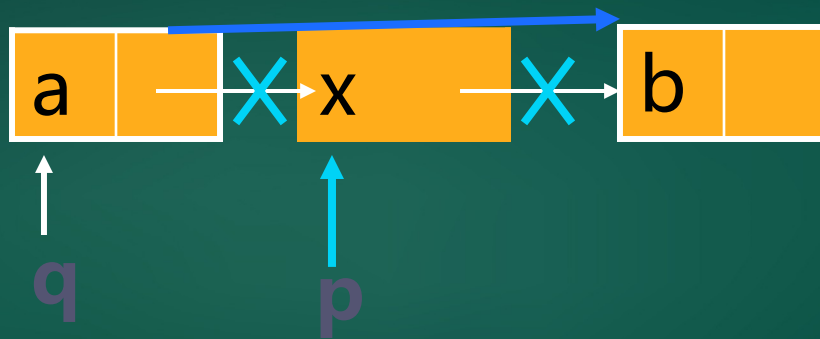
(1) 设定两个指针 p 和 q ， p 指针指向被删除结点， q 为跟踪指针，指向被删除结点的直接前驱结点。

(2) p 从表头指针 $head$ 指向的第一个结点开始依次向后搜索。当 $p \rightarrow data$ 等于 x 时，被删除结点找到。

(3) 修改 p 的前驱结点 q 的指针域。使 p 结点被删除，然后释放存储空间。



单链表的删除运算



2.3 线性表的链式存储结构及其运算

```
void delete(NODE *head,int x)
    /* 删除链表中的给定元素 x*/
{
    NODE *p,*q;
    q=head;
    p=q->next;
    while((p!=NULL)&&(p->data!=x))
        /* 查找要删除的元素 */
    {
        q=p;    p=p->next;
    }
    if(p==NULL)    /* x 结点未找到 */
        printf("%d not found.\n",x);
    else
    {
        q->next=p->next; /* 链接 x 直接后继结点 */
        free(p);         /* 删除 x 结点, 释放 x 结点空间 */
    }
}
```



2.3 线性表的链式存储结构及其运算

5. 输出单链表

若要将单链表按其逻辑顺序输出，就必须从头到尾访问单链表中的每一个结点。

```
void print(NODE *head) /* 输出单向链表各元素 */
{
    NODE *p;
    p=head->next;
    printf("Output the list:");
    while(p!=NULL)
    {
        printf("%3d",p->data);
        p=p->next;
    }
}
```



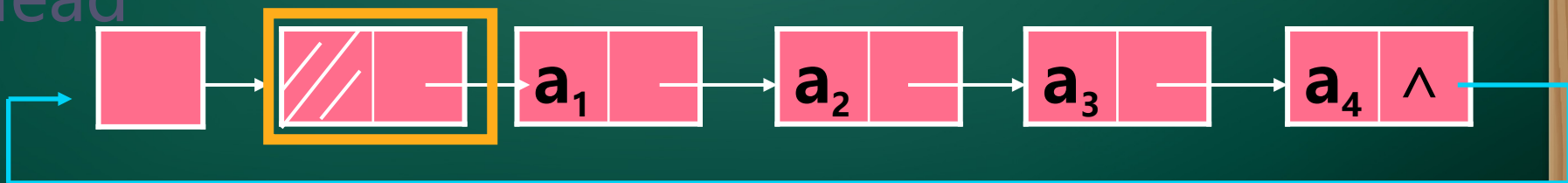
2.3 线性表的链式存储结构及其运算

2.3.3 循环链表结构

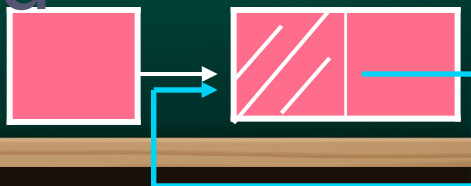
单链表特点：各个节点之间由一个指针域链接，最后一个节点指针域存放的值为 NULL，表示链表的结束。

如果将单链表最后一个结点的指针指向头结点，使链表形成一个环形，此链表就称为循环链表（Circular Link List）。

Head



Head



2.3 线性表的链式存储结构及其运算

循环链表的运算

与单链表基本一致，区别在于最后一个节点的判断，将单链表算法中出现的 NULL 处改为 head 即可。

在很多场合利用循环链表示某些运算比单链表更方便。

例如，查找某一节点的直接前驱。

单链表：必须从 head 指向的头节点出发去找。

循环链表：可从表中任一节点 p 出发去找。



2.3 线性表的链式存储结构及其运算

查某一节点 p 的直接前驱节点

```
#include<stdio.h>
typedef struct node    /* 定义结点的存储结构 */
{   int data;
    struct node *next;
}NODE;
NODE *create_circular()
{   NODE *head,*q,*p; /* 定义指针变量 */
    int a,n;
    head=(NODE*)malloc(sizeof(NODE));
                                /* 建立表头结点 */
    q=head;
    printf("\nInput number of the list: ");
```



2.3 线性表的链式存储结构及其运算

```
scanf("%d",&n); /* 输入单向链表结点个数 */
head->data=n; /* 表头结点赋值 n(表中结点数) */
if(n>0) /* 若 n<=0, 建立仅含表头结点的空表 */
{ printf("Input the list :");
  while(n>0)
  { scanf("%d",&a); /* 输入新元素 */
    p=(NODE*)malloc(sizeof(NODE));
    p->data=a;
    q->next=p;  q=p; n--;
  }
}
q->next=head; /* 循环链表 */
return(head); /* 返回表头指针 head */
}
```



2.3 线性表的链式存储结构及其运算

```
NODE *prior(NODE *p)
{
    NODE *q;
    q=p->next; /* 向后搜索方式查找 */
    while(q->next!=p)
        q=q->next;
    return(q);
}
```



2.3 线性表的链式存储结构及其运算

```
main()                /* 主程序 */
{  NODE *a,*c,*p;
   int i,j;
   a=create_circular();
   printf( "Input j:" );
   scanf( "%d" ,&j); /* 输入要查的节点序号 ,>=0)
   p=a; /* 从头节点搜索序号为j的节点 , 让 p 指向它 */
   for(i=0;i<j;i++)
       p=p->next;      /* 按序号确定一个 p 结点 */
   c=prior(p);
   printf("prior of %d is : %d",p->data,c->data);
}
```



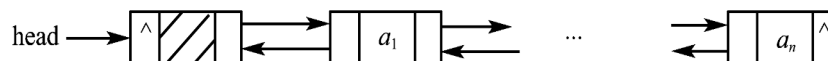
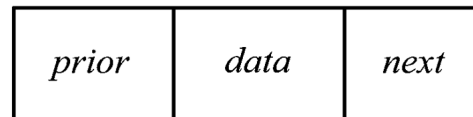
2.3 线性表的链式存储结构及其运算

2.3.4 双向链表结构

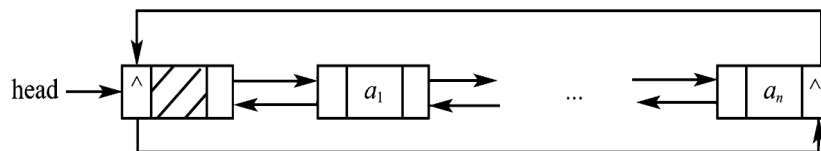
1. 双向链表的基本概念

在单链表的结点中再增加一个指针域，这个指针直接指向该结点的直接前驱。这样，链表中一个结点就有了两个指针域，我们把这样的链表称为双向链表。

如果每条链都构成一个循环链表，则称这样的链表为双向循环链表。



(a)带表头结点的双向链表



(b)带表头结点的双向循环链表



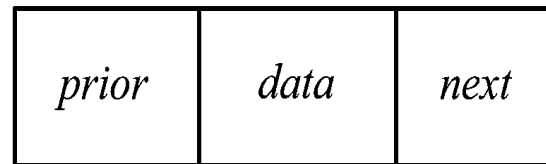
2.3 线性表的链式存储结构及其运算

2.3.4 双向链表结构

2. 双向链表的结构描述

```
typedef struct dupnode /* 结点的存储结构 */  
{ int data;  
  struct node *next,*pior;  
}DUPNODE;
```

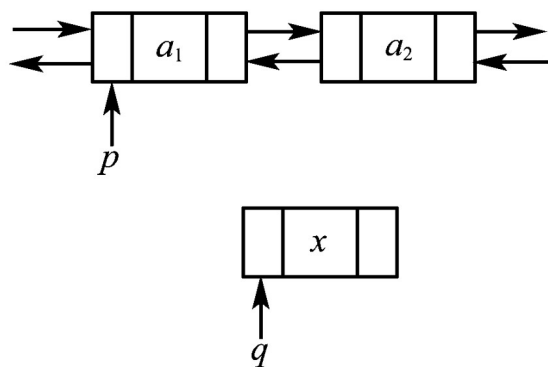
双向链表的运算：如果只涉及一个方向的指针，则运算时与单向链表的算法一致；如果涉及两个方向的指针，则与单链表不同。注意其一个重要特点：若 p 是指向双向链表某一节点的指针，则有：

$$(p \rightarrow next) \rightarrow pior = (p \rightarrow pior) \rightarrow next = p$$


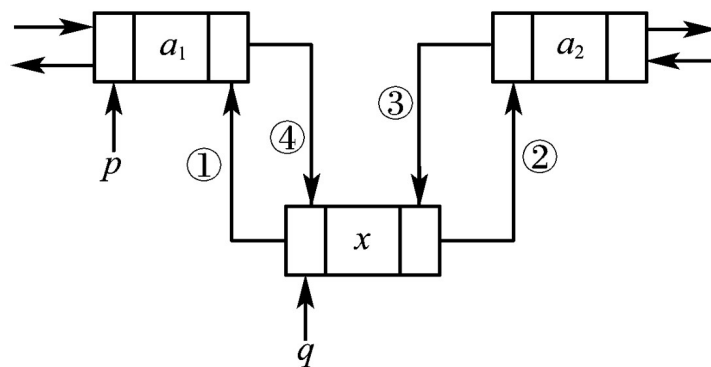
2.3 线性表的链式存储结构及其运算

2. 插入运算

在双向链表的 p 结点之后插入新结点 q 。（修改相应的 4 个指针域的内容）



(a) 插入前



(b) 插入后



2.3 线性表的链式存储结构及其运算

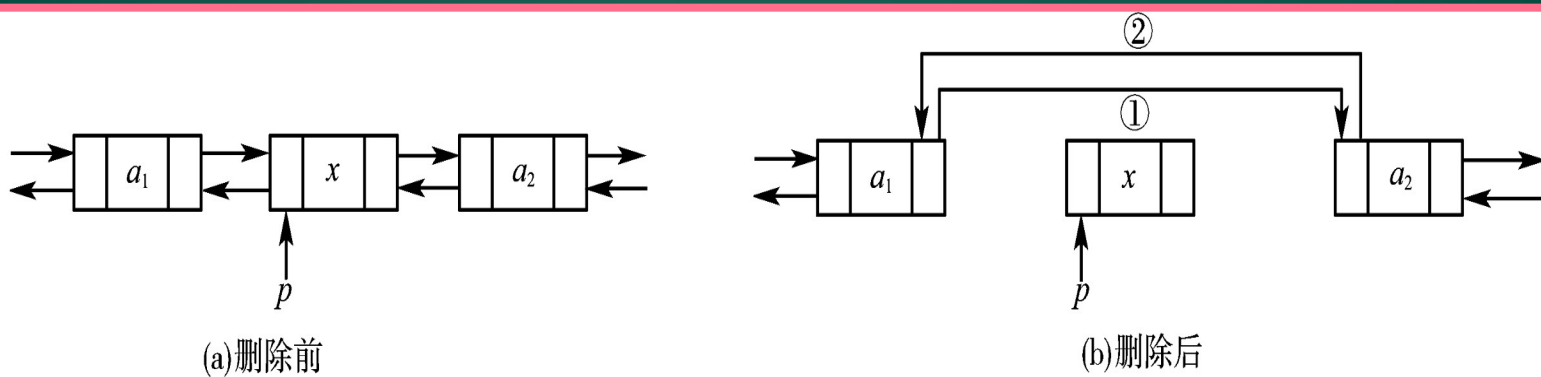
```
void insert ( DUPNODE *p , DUPNODE *q )  
{          /* 把 q 结点插在双向链表的 p 结点之后 */  
    q->prior=p;  
    q->next=p->next;  
    ( p->next ) ->prior=q;  
    p->next=q;  
}
```



2.3 线性表的链式存储结构及其运算

3. 删除运算

将双向链表中的 p 结点删除。（修改相应的 2 个指针域的内容）



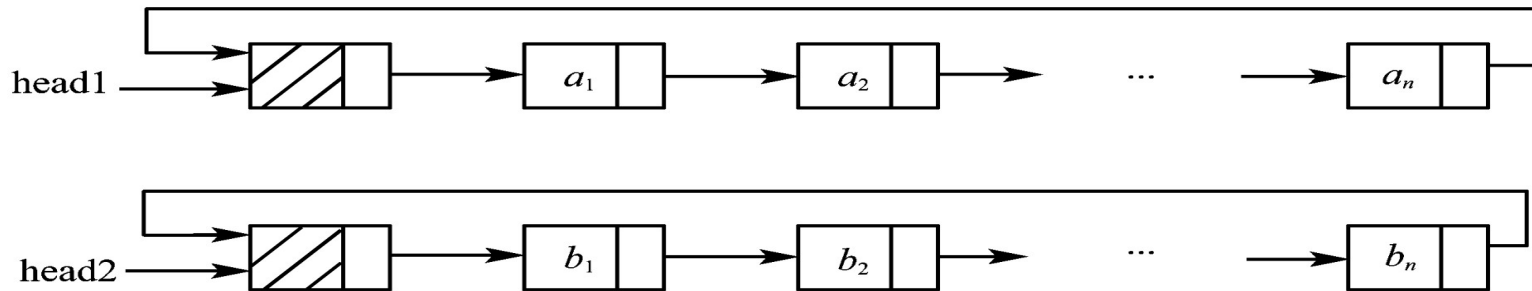
2.3 线性表的链式存储结构及其运算

```
void delete ( DUPNODE *p )  
/* 在双向链表中删除结点 p */  
{  
    ( p->prior ) ->next=p->next;  
    ( p->next ) ->prior=p->prior;  
    free ( p ) ;  
}
```

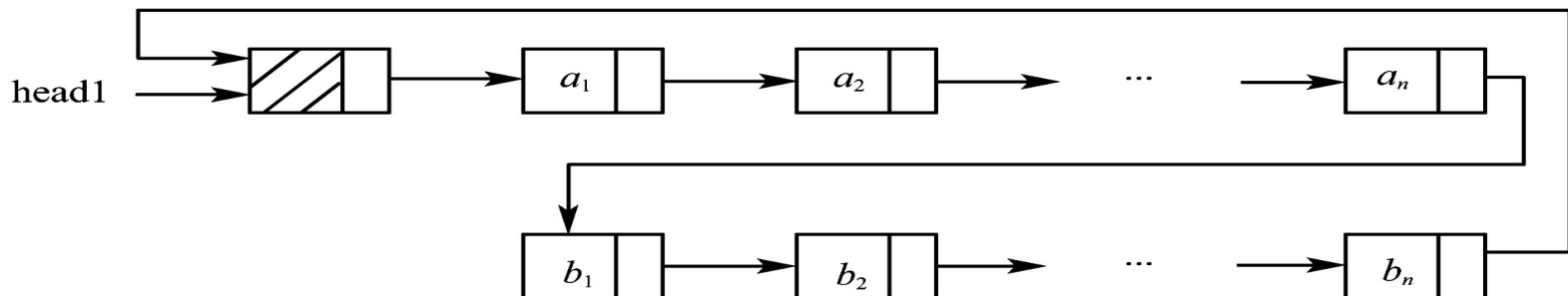


例 2.1 算法应用举例

有两个线性表 A 和 B，都是循环链表存储结构，两个链表头指针分别为 head1 和 head2，将 B 链表链接到 A 链表的后面，合并成



(a) 两个循环链表



(b) 链接后的循环链表

2.4 算法应用举例

```
NODE *connect(NODE *head1,NODE *head2)
{ NODE *p,*q;
  p=head1->next;
  while(p->next!=head1) /* 找 head1 最后一个结点 p*/
    p=p->next;
  q=head2->next;
  while(q->next!=head2) /* 找 head2 最后一个结点 q*/
    q=q->next;
  p->next=head2->next; /* 两表链接 */
  q->next=head1;
  free(head2);          /* 释放 B 表表头结点 */
  return(head1);
}
```



2.4 算法应用举例

例 2.2 一元多项式的加法运算。设有两个一元多项式分别为：

$$A_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (1)$$

$$B_m(x) = b_0 + b_1x + b_2x^2 + \dots + b_mx^m \quad (2)$$

分析：

(1) 式中有 $n+1$ 个系数，可用线性表表示为：

$$A = (a_0, a_1, a_2, a_3, \dots, a_n)$$

(2) 式中有 $m+1$ 个系数，可用线性表表示为：

$$B = (b_0, b_1, b_2, b_3, \dots, b_m)$$

假设 $m \leq n$ ，两个多项式的和 $R_n(x) = A_n(x) + B_m(x)$ 的系数可用线性表表示为：

$$R = (a_0 + b_0, a_1 + b_1, \dots, a_m + b_m, a_{m+1}, \dots, a_n)$$



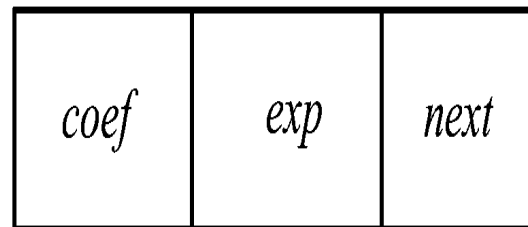
2.4 算法应用举例

方法 1：用线性表的顺序存储结构实现（三个数组存储。

方法 2：用线性表的链式存储结构实现：

多项式中每个**非零项**用一个结点来表示，结点中含有两个数据域和一个指针域，两个数据域分别存放非零项的系数和指数。

```
typedef struct pnode
{   int coef; /* 系数 */
    int exp;  /* 指数 */
    struct pnode *next;
} PNODE;
```



2.4 算法应用举例

设有多项式

$$A(x) = 1 + 2x + 4x^3 \quad (1)$$

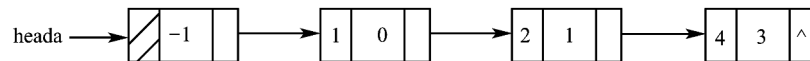
$$B(x) = 2 - 2x + 3x^2 \quad (2)$$

其和为多项式:

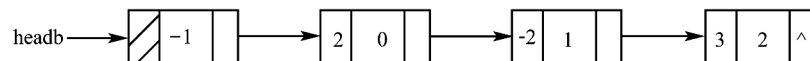
$$R(x) = 3 + 3x^2 + 4x^3 \quad (3)$$

多项式 (1) 加多项式 (2) 的和为
多项式 (3) :

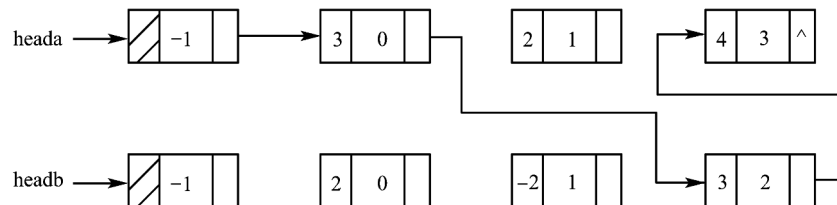
$$R(x) = 3 + 3x^2 + 4x^3 \quad (3)$$



(a) 多项式 $A(x)$



(b) 多项式 $B(x)$



(c) 多项式 $A(x) + B(x)$



本章小结

1. 线性表是一种具有一对一的线性关系的特殊数据结构。
2. 线性表有两种存储方法：用顺序存储方法来表示这种线性关系，得到顺序存储结构（即顺序表）；用链式存储方式来表示这种线性关系，得到线性表的链式存储结构（即链表）。
 2. 线性表的链式存储结构，是通过结点之间的链接而得到的，链式存储结构有单链表、双向链表和循环链表等。
 3. 单链表结点至少有两个域：一个数据域和一个指针域。双向链表结点至少含有三个域：一个数据域和两个指针域。



本章小结

4. 循环链表不存在空指针，最后一个结点的指针指向表头，形成一个首尾相接的环。

5. 为了处理问题方便，在链表中增加一个头结点。

6. 顺序存储可以提高存储单元的利用率，不便于插入和删除运算。链式存储会占用较多的存储空间，可以使用不连续的存储单元，插入、删除运算较方便。

